Lab-4 Assignment

High-Resolution Timing

Objectives

Your objectives in this lab are to:

1. Create a high-resolution timer driver based on the hardware timers built into the STM32F103RB.
2. Use this timer to measure a number of basic arithmetic operations and estimate average and worst-case execution time (WCET).
3. Add a command to your CLI that runs your timing test suite.

Procedure

Phase 1: Timer Driver

Start a fresh project, as you have done in the past. You will be using one of the general-purpose timers. You may use the same timer as in previous labs if you want.

Configure the timer as a continuous, free-running countdown timer. When the timer reaches `0x0000`, it should wrap around to `0xFFFF`. The timer should be driven at the highest available clock driver frequency.

Create two routines that interface with the timer:

- `timer_start()` returns the current value of the timer as a 16-bit signed integer.
- `timer_stop()` takes an `int16_t` obtained from a previous call to `timer_start()` and compares it with the current timer value. It returns the number of timer ticks since the supplied `start_time`.

Your `timer_stop()` function must operate correctly even if the timer wraps around from `0x0000` to `0xFFFF` during the interval. You do not need to handle more than one wraparound. The timer must therefore handle intervals from 0 to 65,535 clocks.

Put your declarations in a file named `timer.h` and your implementation in `timer.c`.

Phase 2: Timing Measurements

Use your timer to measure the average time the processor takes to:

- Add two random 32-bit integers.
- Add two random 64-bit integers.
- Multiply two random 32-bit integers.
- Multiply two random 64-bit integers.
- Divide two random 32-bit integers.
- Divide two random 64-bit integers.
- Copy an 8-byte `struct` using the assignment operator.

- Copy a 128-byte `struct` using the assignment operator.
- Copy a 1024-byte `struct` using the assignment operator.

Hints and Advice

- Each measurement should be the average of 100 trials with random inputs.
- Write functions that generate the random values you need, but do not include random-number generation time in your measurements.
- For the division tests, make sure you do not divide by zero. The test for a zero divisor is part of the measurement.
- Make your measurements with compiler optimization enabled for maximum speed. Try different compiler optimization levels and note the differences in execution time.

Phase 3: CLI Command

Add a new command to your CLI from Lab 2. The command should run the timing test code from Phase 2 and print the results on the screen.

You should be able to capture this output in a terminal program such as Tera Term or PuTTY.

Submission

In your Lab 4 folder, create a `README` that details your measurement times at different compiler optimization levels.